

Brno University of Technology
Faculty of Information Technology

Image Processing (in English) (ZPOe 25/26)

Application of Basic Image Processing Filters on Human Face Images

Project Report

Adar Otieno	`xotiena00`
Deniz Aksoy	`xaksoyd00`
Eyobed Awel Nuri	`xnuriey00`

May 9, 2026

Contents

1 Introduction **3**

2 Compilation and Running **3**

3 Implementations **3**

3.1 Eyobed's Filters . . . 3

3.1.1 Original Input Images . . . 3

3.1.2 Histogram Equalization . . . 4

3.1.3 Lens Distortion . . . 4

3.1.4 Posterize . . . 5

3.1.5 Vignette Blur . . . 6

3.1.6 Discussion . . . 7

3.2 Adar's Filters . . . 7

3.3 Deniz's Filters . . . 7

3.3.1 Original Input Images . . . 7

3.3.2 Threshold Filter . . . 8

3.3.3 Diffuse Filter . . . 9

3.3.4 Crosshatch Filter . . . 10

3.3.5 Hue Rotate Filter . . . 11

3.3.6 Evaluation and Discussion . . . 12

4 Conclusion **12**

1 Introduction

The students have decided to rework the project by removing previously proposed trivial filters such as sharpen and replaced them with more sophisticated ones as per the suggestions of the professor. We developed 12 filters from different categories in total, 4 filters per student.

2 Compilation and Running

The project uses CMake for build management. Standard compilation and execution steps:

```
mkdir build && cd build
cmake .. && make
./project
```

The application expects input images in the `data/` directory. The GUI features trackbars for image/filter selection and a side-by-side visualization window (800x600) with a metadata panel for filter descriptions. If you can not see the windows, drag them around to expose them.

3 Implementations

This section presents the filters developed by each student. Each student individually developed 4 filters, only the GUI is shared by the students.

3.1 Eyobed's Filters

3.1.1 Original Input Images

The following four images (Figure 1) serve as the baseline for all subsequent filter demonstrations in this section.



(a) Source 1 (b) Source 2 (c) Source 3 (d) Source 4

Figure 1: Original unprocessed input images used for Eyobed's filter evaluations.

3.1.2 Histogram Equalization

Histogram Equalization enhances image contrast by redistributing the brightness values. To prevent unwanted color shifting, the algorithm operates in the YCrCb color space, applying the equalization only to the luminance channel before converting back to BGR.

The algorithm calculates the histogram frequencies, the Probability Density Function (PDF), and the Cumulative Distribution Function (CDF). The PDF for a pixel intensity i is calculated as:

$$PDF(i) = \frac{\text{count}(i)}{\text{total pixels}}$$

The CDF is then calculated as the cumulative sum of the PDF:

$$CDF(i) = \sum_{j=0}^{i} PDF(j)$$

Finally, the new remapped pixel value is computed by scaling the CDF to the 0-255 range:

$$\text{Output}(x, y) = \text{round}(CDF(I(x, y)) \times 255)$$



(a) Result 1 (b) Result 2 (c) Result 3 (d) Result 4

Figure 2: Results after applying the Histogram Equalization filter.

3.1.3 Lens Distortion

The filter uses an inverse geometric mapping to simulate lens distortion. For each output pixel (x, y), the source coordinate (s_x, s_y) is calculated through three steps listed below:

1. Normalization: Coordinates are centered and scaled by $R = \sqrt{c_x^2 + c_y^2}$:

$$d_x = \frac{x - c_x}{R}, \quad d_y = \frac{y - c_y}{R}$$

2. Distortion: A radial factor is computed using coefficient k:

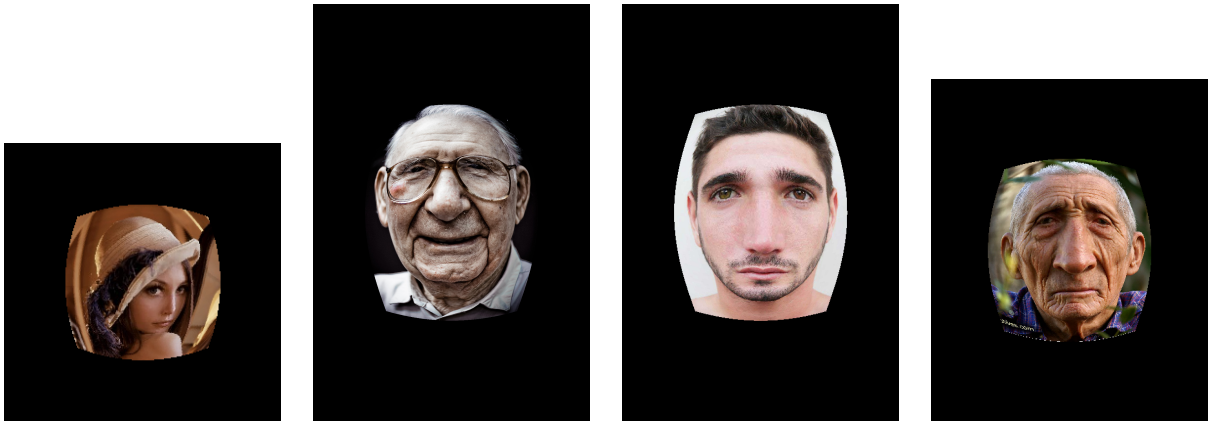
$$\text{dist} = 1 + k(d_x^2 + d_y^2)$$

Positive k produces barrel distortion, while negative k produces pincushion effects.

3. Mapping: Coordinates are scaled back to source pixel space:

$$s_x = d_x \cdot \text{dist} \cdot R + c_x, \quad s_y = d_y \cdot \text{dist} \cdot R + c_y$$

Pixels are sampled using nearest-neighbor interpolation. The computational complexity is $O(W \times H)$.



(a) Result 1 (b) Result 2 (c) Result 3 (d) Result 4

Figure 3: Results after applying the Lens Distortion filter ($k = 5.0$).

3.1.4 Posterize

Reduces color depth by quantizing each channel into $L = 8$ discrete levels. A 256-entry LUT is computed once to improve performance.

1. Step Calculation: The spacing between levels:

$$\text{step} = \frac{255}{L - 1} \approx 36.4$$

2. LUT Construction: Each intensity i is snapped to the nearest level index, then scaled back:

$$\text{level}_i = \text{round}\left(\frac{i}{\text{step}}\right), \quad \text{LUT}[i] = \text{round}(\text{level}_i \times \text{step})$$

3. Pixel Remapping: Each BGR channel is independently mapped:

$$\text{Output}_c(x, y) = \text{LUT}[\text{Input}_c(x, y)], \quad c \in \{B, G, R\}$$

Complexity: $O(W \times H)$ for the pixel pass, plus $O(256)$ for LUT construction.



(a) Result 1 (b) Result 2 (c) Result 3 (d) Result 4

Figure 4: Results after applying the Posterize filter ($L = 8$ levels).

3.1.5 Vignette Blur

Softens edges while keeping the center sharp. Implemented as a two step process: a 15×15 box blur followed by a radial blend.

1. **Box Blur:** For interior pixels (7-pixel border excluded), compute the 15×15 mean kernel convolution.

$$\text{Blur}(x, y) = \frac{1}{225} \sum_{dy=-7}^{7} \sum_{dx=-7}^{7} \text{Input}(x + dx,\ y + dy)$$

2. **Radial Mask:** Normalize the distance from center (c_x, c_y) by the corner radius R:

$$d = \frac{\sqrt{(x - c_x)^2 + (y - c_y)^2}}{R}, \quad d \in [0, 1]$$

The center maps to 0 (no blur) and corners to 1 (full blur).

3. **Blending:** Linearly interpolate between original and blurred using d as the weight:

$$\text{Output}(x, y) = \text{Input}(x, y) \cdot (1 - d) + \text{Blur}(x, y) \cdot d$$

Complexity: $O(W \times H)$ since the blur and blend are computed in a single pass.



(a) Result 1 (b) Result 2 (c) Result 3 (d) Result 4

Figure 5: Results after applying the Vignette Blur filter using a 15x15 manual kernel.

3.1.6 Discussion

The author of this section studied about implementation of histogram eqalization,lens distortion, posterization, and vignette blur filters. He believes that these filters are good demonstrations of concepts discussed in class. Additional techniques like look up tables helped to take into consideration the performance aspect of image processing.

3.2 Adar's Filters

Adar's contributions focus on manual pixel manipulation using native loops, avoiding optimized OpenCV functions for educational purposes.

- **Relief Emboss:** Creates a 3D gray relief texture by subtracting the bottom-right neighboring pixel from the top-left neighbor and adding an offset of 128 to center the result.
- **Dodge:** Implements the classic Color Dodge blend formula: `(pixel * 255) / max(1, 255 - pixel)`. This blows out the highlights and drastically increases contrast.
- **Swirl:** Warps the image into a vortex by converting Cartesian coordinates to Polar coordinates, applying a nonlinear rotational offset based on the radius, and converting back to sample the pixel.
- **Bloom:** Extracts bright pixels (intensity > 180) and applies a brute-force nested 7x7 loop to blur them. The resulting glow is then added directly back to the original image.

3.3 Deniz's Filters

Similar to Adar, Deniz's filters prioritize foundational image processing algorithms implemented through direct pixel-by-pixel matrix iteration. The objective of these filters is not only to produce visually interesting effects but also to demonstrate the mathematical principles behind image segmentation, stochastic distortion, procedural rendering, and color-space manipulation.

3.3.1 Original Input Images

The following images were selected as the common input dataset for evaluating the visual behavior and robustness of the implemented filters.



(a) Input 1 (b) Input 2 (c) Input 3 (d) Input 4

Figure 6: Original input images used for evaluating Denis's image processing filters.

3.3.2 Threshold Filter

The Threshold filter is a classical image segmentation and binarization technique used to separate foreground information from the background. The implementation manually iterates through every pixel and computes its luminosity value using the weighted grayscale conversion formula:

$$L = 0.299R + 0.587G + 0.114B$$

This weighted grayscale formula reflects the sensitivity of the human visual system, which perceives green wavelengths more strongly than blue wavelengths. After computing the luminosity, a binary threshold operation is applied:

$$\text{Output} = \begin{cases} 255, & \text{if } L > T \\ 0, & \text{otherwise} \end{cases}$$

where T denotes the threshold value.

Thresholding belongs to the category of binary image processing and is widely used in OCR systems, edge detection, document analysis, license plate recognition, and preprocessing pipelines for machine learning applications.

The computational complexity of the algorithm is:

$$O(W \times H)$$

where W and H represent the width and height of the image respectively.



(a) Result 1 (b) Result 2 (c) Result 3 (d) Result 4

Figure 7: Results after applying the Threshold filter to the original images shown in Figure 6.

3.3.3 Diffuse Filter

The Diffuse filter creates a frosted-glass distortion effect by replacing each pixel with a randomly selected neighboring pixel located within a predefined radius. Unlike traditional blur filters that average neighboring pixels, the diffuse filter uses stochastic sampling to introduce randomized local distortion.

For a given pixel coordinate (x, y), a neighboring pixel is selected according to:

$$(x', y') = (x + r_x,\ y + r_y)$$

where r_x and r_y are random offsets constrained within the selected radius.

This process creates an organic scattering effect that resembles viewing an object through textured or frosted glass. The filter belongs to the category of artistic image stylization and non-linear image distortion.

The algorithm is computationally linear:

$$O(W \times H)$$

although random memory access patterns may slightly reduce cache efficiency compared to standard convolution-based methods.



(a) Result 1 (b) Result 2 (c) Result 3 (d) Result 4

Figure 8: Results after applying the Diffuse filter to the original images shown in Figure 6.

3.3.4 Crosshatch Filter

The Crosshatch filter is a non-photorealistic rendering (NPR) technique designed to imitate traditional pen-and-ink sketching methods. Instead of relying on convolution kernels, the filter procedurally generates diagonal line structures using modulo arithmetic on pixel coordinates.

The basic pattern generation can be represented as:

$$(x + y) \bmod n = 0$$

where n controls the spacing between hatch lines.

Additional line layers are introduced depending on the darkness of the underlying pixel. Darker image regions receive denser hatch structures, while brighter regions contain fewer strokes. This behavior simulates classical artistic shading techniques used in traditional crosshatching.

The filter is computationally lightweight and maintains linear complexity:

$$O(W \times H)$$

Crosshatching is frequently used in sketch simulation, comic-book rendering, procedural graphics, and artistic visualization systems.



(a) Result 1 (b) Result 2 (c) Result 3 (d) Result 4

Figure 9: Results after applying the Crosshatch filter to the original images shown in Figure 6.

3.3.5 Hue Rotate Filter

The Hue Rotate filter modifies image colors by rotating the Hue channel in the HSV color space. The image is first converted from BGR to HSV using OpenCV before manually modifying the Hue channel.

The hue transformation is computed as:

$$H' = (H + \Delta H) \bmod 180$$

where $\Delta H = 60$ degrees in the implemented effect.

Modulo arithmetic is required because hue values form a circular color wheel. Once the hue exceeds the maximum range, it wraps around to the beginning of the spectrum.

Hue rotation is widely used in artistic color grading, game graphics, data augmentation, augmented reality filters, and image stylization pipelines.

The computational complexity remains:

$$O(W \times H)$$

since each pixel is processed independently.



(a) Result 1 (b) Result 2 (c) Result 3 (d) Result 4

Figure 10: Results after applying the Hue Rotate filter to the original images shown in Figure 6.

3.3.6 Evaluation and Discussion

The implemented filters successfully demonstrate several fundamental concepts in image processing, including binary segmentation, stochastic pixel manipulation, procedural non-photorealistic rendering, and color-space transformation. The Threshold filter proved highly effective for simplifying image structures and emphasizing high-contrast regions, making it particularly suitable for preprocessing tasks such as OCR and edge extraction. The Diffuse filter generated visually organic distortions that simulated frosted-glass effects through randomized neighborhood sampling, although fine image details were partially degraded due to stochastic pixel replacement. The Crosshatch filter effectively transformed photographic content into stylized sketch-like renderings using lightweight mathematical coordinate patterns, demonstrating how procedural line generation can emulate traditional artistic shading techniques. Finally, the Hue Rotate filter showed that manipulating the HSV color space provides an efficient method for altering image appearance while preserving geometric structure and brightness information. Overall, the results confirm that relatively simple pixel-level algorithms can produce both technically meaningful and visually compelling transformations while maintaining computational efficiency with linear time complexity.

4 Conclusion

This project successfully demonstrates the application of both fundamental and advanced image processing techniques. The students also estimated the computational complexity of each filter thanks to the knowledge they acquired in the course TINe. Moreover, we also learned the importance of using appropriate data structures and algorithms for image processing. The authors would like to acknowledge the utility of AI tools such as ChatGPT in the realization of this project in a reasonable and responsible manner.

References

[1] Ma, L., Chen, Y., & Moore, K. L. (2003). *A New Analytical Radial Distortion Model for Camera Calibration*. arXiv preprint cs/0307046.